il compose.override.yaml verrebbe generato (magari a partire da un template) da 40.services.sh (prima di eseguire docker compose) in base ai servizi specificati nella riga di comando ?

Sì, può farlo — ma ti sconsiglio di chiamarlo `compose.override.yaml` e di sovrascriverlo a ogni invocazione. È più robusto generare un **file Compose transitorio specifico per l'invocazione**, ad esempio `.docker-dev/generated/compose.services.yaml`, e passarlo come ultimo `-f`. Compose merge i file nell'ordine in cui vengono forniti: i successivi aggiungono o sovrascrivono i precedenti. [1] [2]

Perché non `compose.override.yaml`

`compose.override.yaml` ha un significato convenzionale: se esiste accanto a `compose.yaml`, Compose lo carica automaticamente insieme al file base. [1]

Se `40.services.sh` lo genera dinamicamente, ottieni questi problemi:

- Un utente che vuole personalizzare il proprio ambiente perde il file o vede le proprie modifiche sovrascritte.
- Non è chiaro se il file rappresenta "preferenze del progetto" oppure "servizi richiesti in questo comando".
- Due esecuzioni concorrenti con servizi diversi si sovrascrivono a vicenda.
- Il file cambia anche quando non dovrebbe, complicando caching, hash di configurazione e debug.

Layout consigliato

```
Python/foo/.docker-dev/
├── compose.yaml                    # generato da apply-templates.sh
├── compose.user.yaml               # opzionale, modificato dall'utente
├── .env                            # opzionale, locale
└── generated/
    └── compose.services.yaml       # generato da 40.services.sh
```

`docker-dev.sh` costruisce sempre la catena in ordine crescente di priorità:

```
docker compose \
  -p dev.python.foo \
```

```
  --project-directory "$PROJECT_DIR" \
  -f .docker-dev/compose.yaml \
  -f .docker-dev/compose.user.yaml \
  -f .docker-dev/generated/compose.services.yaml \
  up -d
```

Così:

1. `compose.yaml`: template del linguaggio, servizio `dev`, volumi base.
2. `compose.user.yaml`: personalizzazioni dell'utente/progetto.
3. `compose.services.yaml`: servizi scelti sulla riga di comando, con la priorità più alta.

Cosa genera `40.services.sh`

Il suo compito non è necessariamente generare YAML per ogni servizio. Due strategie, utilizzabili insieme:

1. Profili per servizi già dichiarati

Se PostgreSQL, Redis, Mailpit ecc. sono già definiti in `compose.yaml` o in un file dei servizi, `40.services.sh` non deve generare nulla: calcola solo i profili.

```
docker-dev --target=Python/foo --with=db,cache
```

diventa:

```
docker compose \
  -p dev.python.foo \
  -f .docker-dev/compose.yaml \
  --profile db \
  --profile cache \
  up -d
```

I profili sono il meccanismo Compose pensato per attivare selettivamente servizi; i servizi senza profilo partono sempre, quelli con profilo solo quando richiesto. [2] [3]

2. File generato per servizi dinamici

Genera YAML solo quando il servizio dipende da parametri della riga di comando: porta host, versione immagine, credenziali, nome volume, flag extra, percorsi o dipendenze.

Esempio per:

```
docker-dev --target=Python/foo \
  --service db=postgres:16,port=15432 \
  --service cache=redis:7
```

`40.services.sh` può produrre:

```
# generated/compose.services.yaml
services:
  db:
    image: postgres:16
    profiles: ["db"]
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: app
      POSTGRES_DB: app
    ports:
      - "127.0.0.1:15432:5432"
    volumes:
      - db-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d app"]
      interval: 5s
      timeout: 3s
      retries: 10

  cache:
    image: redis:7
    profiles: ["cache"]

  dev:
    environment:
      DATABASE_URL: postgres://app:app@db:5432/app
      REDIS_URL: redis://cache:6379/0
    depends_on:
      db:
        condition: service_healthy
      cache:
        condition: service_started

volumes:
  db-data:
    name: db.dev.python.foo
```

Poi `docker-dev.sh` invoca:

```
docker compose \
  -p dev.python.foo \
  -f .docker-dev/compose.yaml \
  -f .docker-dev/generated/compose.services.yaml \
  --profile db --profile cache \
  up -d
```

Attenzione al merge

Il merge Compose non è sempre “append”: per mapping aggiunge o unisce le chiavi, ma per alcuni attributi — tra cui `command`, `entrypoint` e `healthcheck.test` — l’ultimo file **sostituisce** il valore precedente.[4] [5]

Questo è rilevante per il servizio `dev`: se il file dei servizi aggiunge `environment`, va bene; ma se definisce `entrypoint` o `command`, sovrascrive quello del template Python. Quindi:

- Nel file dei servizi, modifica `dev` solo per aggiungere variabili d'ambiente, mount o `depends_on`.
- Non ridefinire `entrypoint`, `command`, `user` o `working_dir` a meno che sia esplicitamente intenzionale.
- Se devi sostituire interamente una lista, usa `!override`; per azzerarne una, `!reset`.[5]

Contract fra script

Ti suggerirei questo contratto semplice:

```
40.services.sh
  input:
    - target: Python/foo
    - project name: dev.python.foo
    - requested services/profiles
    - UID/GID host
    - project directory

  output:
    - .docker-dev/generated/compose.services.yaml
    - required profiles, one per line or comma-separated
    - exit code 0/1

docker-dev.sh
  responsibility:
    - create generated/ directory
    - run 40.services.sh
    - collect profiles
    - compose final -f list
    - invoke docker compose
```

Esempio minimale in Bash:

```
services_file="$PROJECT_DIR/.docker-dev/generated/compose.services.yaml"
profiles_file="$PROJECT_DIR/.docker-dev/generated/profiles"

rm -f "$services_file" "$profiles_file"
mkdir -p "$(dirname "$services_file")"

langs/Python/bin/40.services.sh \
  --target "$TARGET" \
  --project-name "$PROJECT_NAME" \
  --with "$DDEV_WITH" \
  --services-file "$services_file" \
  --profiles-file "$profiles_file"

compose_args=(
  -p "$PROJECT_NAME"
  --project-directory "$PROJECT_DIR"
```

```
  -f "$PROJECT_DIR/.docker-dev/compose.yaml"
)

[[ -f "$PROJECT_DIR/.docker-dev/compose.user.yaml" ]] && \
  compose_args+=(-f "$PROJECT_DIR/.docker-dev/compose.user.yaml")

[[ -s "$services_file" ]] && \
  compose_args+=(-f "$services_file")

while IFS= read -r profile; do
  [[ -n "$profile" ]] && compose_args+=(--profile "$profile")
done < "$profiles_file"

docker compose "${compose_args[@]}" up -d
```

Regola pratica

Usa questa gerarchia:

Esigenza	Meccanismo
Servizio standard, configurazione fissa	Profilo in `compose.yaml`
Servizio standard con porta/credenziali variabili	Template + variabili in `.env`
Servizio interamente definito dalla CLI	`generated/compose.services.yaml`
Preferenze persistenti dell'utente	`compose.user.yaml`
Override temporaneo di debug	File temporaneo passato come ultimo `-f`

Quindi: **sì,** `40.services.sh` **può generare un override**, ma fallo come file generato e transitorio, non come `compose.override.yaml`. Riserva `compose.override.yaml` — o meglio `compose.user.yaml`, per evitare l'auto-load implicito — alle personalizzazioni stabili del progetto o dell'utente.[2] [1]

⁂

1. https://docs.docker.com/compose/how-tos/multiple-compose-files/merge/
2. https://docs.docker.com/reference/cli/docker/compose.md
3. https://docs.docker.com/compose/how-tos/environment-variables/envvars/
4. https://github.com/Krishnamurtyp/docker-compose-spec/blob/master/13-merge.md
5. https://github.com/compose-spec/compose-spec/blob/main/13-merge.md
6. https://oneuptime.com/blog/post/2026-01-30-docker-compose-profiles/view
7. https://full-chaos.github.io/container-compose/current/site/tutorials/profiles/
8. https://docs.docker.jp/v20.10/compose/profiles.html
9. https://docker.ubitools.com/compose/how-tos/profiles/
10. https://docs.docker.com/compose/how-tos/profiles/
11. https://docs.docker.com/reference/compose-file/merge/

12. https://docs.docker.com/compose/how-tos/multiple-compose-files/extends/
13. https://docs.docker.com/compose/how-tos/multiple-compose-files.md
14. https://learn.microsoft.com/en-us/dotNET/architecture/microservices/multi-container-microservice-net-applications/multi-container-applications-docker-compose
15. https://lours.me/posts/compose-tip-024-profiles/